

Documentation

Wireless Sensor Monitoring and Alert System

Group A — Team A6
07.07.2026

1 Team Members

Member	Role	Responsibilities
Daniel Chidi Chimezie (1230515)	ESP32 Developer	KY-013 sensor test, ESP32 sensor + MQTT integration, ESP32 BLE server, Pi OS + Mosquitto setup, Flow 01, Requirements Diagram (SysML)
Stephen Uzochi Obuh (2210044)	Network Developer	KY-031 sensor test, WiFi connection test, Arduino BLE reconnect test, Node.js + Node-RED install, Flow 02, Use Case Diagram + State Machine Diagram (SysML)
Riwaj Ghimire (1232171)	Arduino / Dashboard Developer	KY-016 sensor test, Arduino actuator integration, Arduino BLE client, Node-RED config, Flow 03 + Flow 04, Block Definition Diagram (SysML)
Bertrand Mungu Cho (2220052)	Integration & QA	KY-012 sensor test, threshold logic test, wiring diagrams, end-to-end system test, Flow 05 + alert log, Sequence Diagram (SysML)

2 Introduction

The Internet of Things (IoT) describes the network of physical devices embedded with sensors, software, and wireless connectivity that enables them to collect and exchange data autonomously. A Wireless Sensor Network (WSN) is a practical IoT subsystem in which spatially distributed sensor nodes monitor physical or environmental conditions and relay measurements to a central processing point without physical wiring.

This project implements a Wireless Sensor Monitoring and Alert System targeting environments where temperature anomalies and physical disturbances (knocks or vibrations) must be detected and acted upon in real time. Representative application domains include server room monitoring, industrial equipment surveillance, smart building security, and laboratory environment control.

The prototype consists of three hardware nodes: an ESP32 that reads sensors and bridges BLE and WiFi/MQTT; an Arduino Uno WiFi Rev2 that drives local visual and audio actuators via BLE; and a Raspberry Pi 3 that hosts the MQTT broker and a web-based Node-RED dashboard with remote control capabilities.

3 Concept Description

The system is a Smart Laboratory Safety and Emergency Alert System. It monitors temperature and physical shock (knock) events in real time and reacts through local actuators and a remote dashboard. The main application is continuous environment monitoring with graded alerts: NORMAL, WARNING, and CRITICAL, each triggering distinct LED colours and buzzer patterns on the Arduino, and visible on the Node-RED dashboard.

Block Definition Diagram

The Block Definition Diagram (BDD) shows the hardware composition of the system. The top-level system block decomposes into: the microcontroller tier (Arduino Uno WiFi Rev2 and ESP32 DevKitC V4), the input sensor tier (KY-013 NTC thermistor and KY-031 knock sensor), the control output tier (KY-016 RGB LED and KY-012 active buzzer), and the Raspberry Pi 3 tier running Mosquitto MQTT broker and Node-RED dashboard.

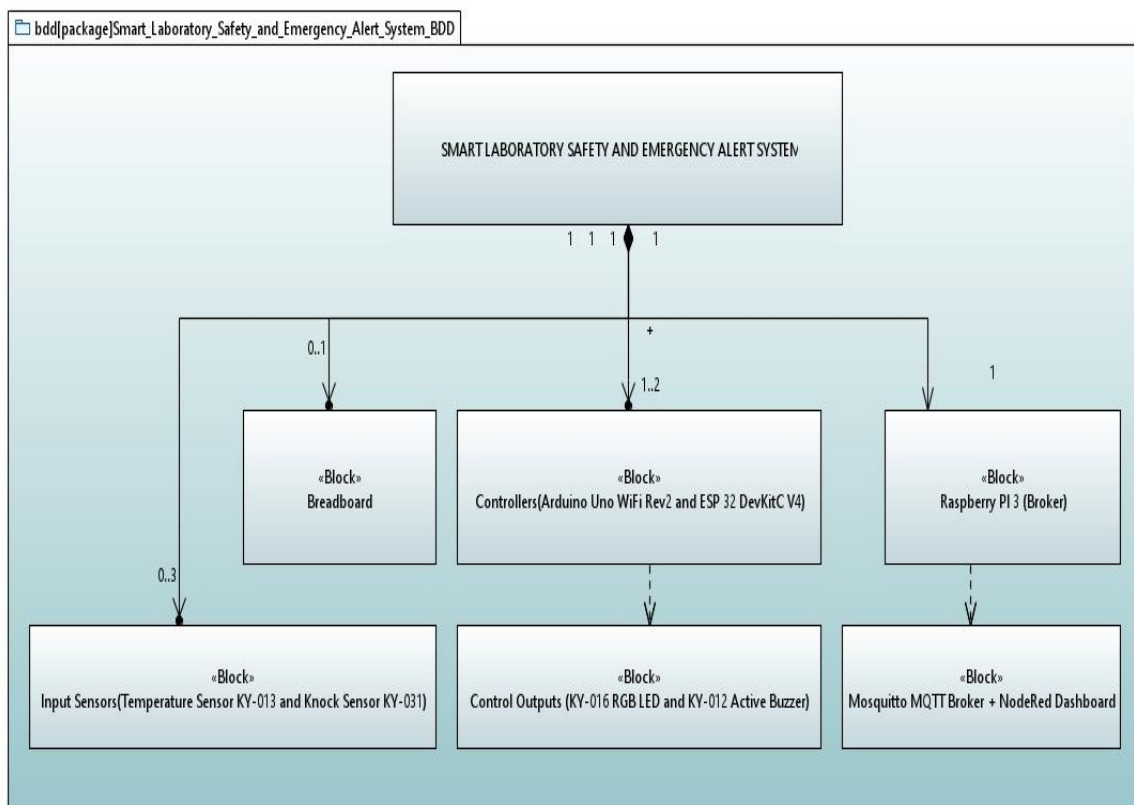


Figure 1: Block Definition Diagram (hardware composition of the system)

Requirements Diagram

The Requirements Diagram captures what the system must do (functional) and how well it must do it (non-functional). Functional requirements include ADC signal conversion (FR-001), audible and visual alerts (FR-002), knock detection (FR-003), temperature monitoring (FR-005), and data evaluation with MQTT publishing and remote command handling (FR-006). Non-functional requirements cover safety response time (5 s), data accuracy, scalability, WiFi security, energy efficiency, maintainability, and 24/7 reliability.

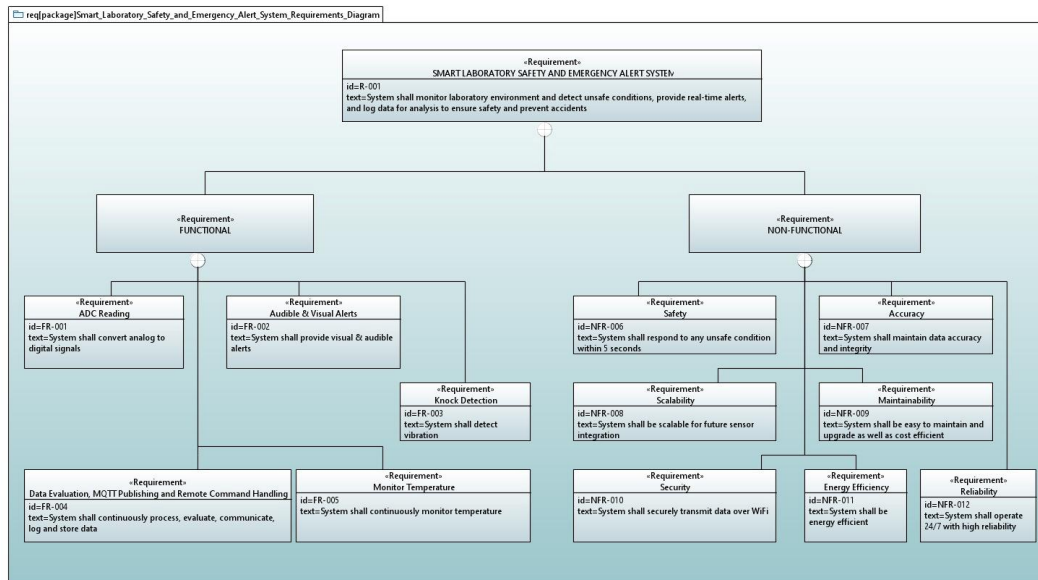


Figure 2: Requirements Diagram (functional and non-functional requirements)

Devices, Sensors, and Actuators

Component	Module	Board / Pin	Interface
Temperature Sensor	KY-013 NTC Thermistor	ESP32 GPIO34 (VP)	Analog ADC 12-bit
Knock / Vibration Sensor	KY-031	ESP32 GPIO32	Digital INPUT_PULLUP
RGB LED	KY-016	Arduino D9 / D10 / D11	Digital OUTPUT
Active Buzzer	KY-012	Arduino D6	Digital / tone()
MQTT Broker	Mosquitto 2.x	Raspberry Pi 3, port 1883	WiFi TCP
Dashboard	Node-RED + node-red-dashboard	Raspberry Pi 3, port 1880	HTTP / WebSocket

Communication Overview

- BLE: ESP32 advertises a custom GATT service (UUID 19B10000-A8F2-537E-4F6C-D104768A1214). The notify characteristic carries the alert status as 1-2 ASCII bytes: '0'=NORMAL, '1'=WARNING, '2'=CRITICAL; 'M' prefix indicates muted.
- WiFi / MQTT: ESP32 publishes a semicolon-delimited packet to topic sensors/node01 every ~2 seconds. Node-RED sends commands back on sensors/node01/cmd.

• Packet example:
TEMP:23.4;KNOCK:2;STATUS:WARNING;MUTE:0;WARN:25.0;CRIT:30.0

4 Project / Team Management

The project followed an incremental development methodology divided into four sequential phases. Each phase had a defined entry condition (previous phase passing its tests) and explicit pass criteria, minimising integration risk. Version control and task ownership were tracked per member throughout the project.

Task Breakdown

Member	Section 1 Sensor Tests	Section 2 Integration	Section 3 BLE	Section 4 Pi & Node-RED	SysML Diagrams
Daniel Chidi Chimezie (1230515)	KY-013 temp test	ESP32 sensors + MQTT test	ESP32 BLE server	Tasks 1-3 + Flow 01	Requirements Diagram
Stephen Uzochi Obuh (2210044)	KY-031 knock test	WiFi connection test	BLE reconnect test	Tasks 4-6 + Flow 02	Use Case Diagram + State Machine Diagram
Riwaj Ghimire (1232171)	KY-016 RGB test	Arduino actuators	Arduino BLE client	Task 7 + Flow 03 + Flow 04	Block Definition Diagram
Bertrand Mungu Cho (2220052)	KY-012 buzzer test	Threshold logic test	Wiring + integration	Task 8 + Flow 05	Sequence Diagram

Each member owned their tasks end-to-end: writing the code, flashing the hardware, running the defined pass tests, and being prepared to explain implementation decisions during the Q&A.

5 Technologies

5.1 Sensors

KY-013 NTC Thermistor: Connected to ESP32 GPIO34 (input-only VP pin) via a 10 kOhm voltage divider. The 12-bit ADC (0–4095) measures the mid-point voltage. Resistance is derived from the ADC ratio and temperature converted using the Steinhart-Hart equation with B-coefficient 3950 K and reference resistance 10 kOhm at 25°C (298.15 K).

KY-031 Knock Sensor: Piezoelectric vibration switch on GPIO32 with INPUT_PULLUP. The signal goes LOW on impact. Falling-edge detection with 50 ms hardware debounce counts events. WARNING: ≥ 2 knocks within 2 s; CRITICAL: ≥ 5 knocks within 5 s. Status held for 5 s after the last qualifying event.

5.2 Communication Protocols

Bluetooth Low Energy (BLE): The ESP32 operates as a GATT server using the ESP32 BLE Arduino library. A custom 128-bit service UUID hosts a single characteristic with READ and NOTIFY properties. The Arduino uses the ArduinoBLE library as GATT client, scanning for the service UUID, subscribing to notifications, and reacting within milliseconds of any state change.

MQTT over WiFi: The ESP32 uses the PubSubClient library to connect to a Mosquitto broker on the Raspberry Pi (port 1883). Sensor data is published to sensors/node01 at QoS 0. Commands from the dashboard are received on sensors/node01/cmd. The broker allows anonymous connections for the lab environment.

5.3 Programming Languages and Libraries

- C++ (Arduino framework) — ESP32 and Arduino sketches
- JavaScript — Node-RED function nodes for packet parsing and command building
- Bash — Raspberry Pi setup, Mosquitto configuration, CLI testing
- ESP32 BLE Arduino — BLE GATT server on ESP32
- ArduinoBLE — BLE GATT client on Arduino Uno WiFi Rev2
- PubSubClient — MQTT publish/subscribe on ESP32
- WiFi.h — WiFi connectivity on ESP32
- node-red-dashboard — UI widgets for Node-RED (gauge, chart, slider, switch, button, text)

6 Implementation

This section describes the static structure and runtime behaviour of the system using SysML diagrams, followed by descriptions of each component's implementation.

State Machine Diagram

The State Machine Diagram models the system's operational lifecycle. After initialisation the system enters the Normal state and continuously monitors sensor inputs. An abnormal condition (temperature above warning threshold or knock event detected) transitions the system to Warning. If the warning threshold is further exceeded, the system moves through Alert to Emergency. A false alarm returns the system to Warning; when conditions are safe again, Recovery transitions back to Normal.

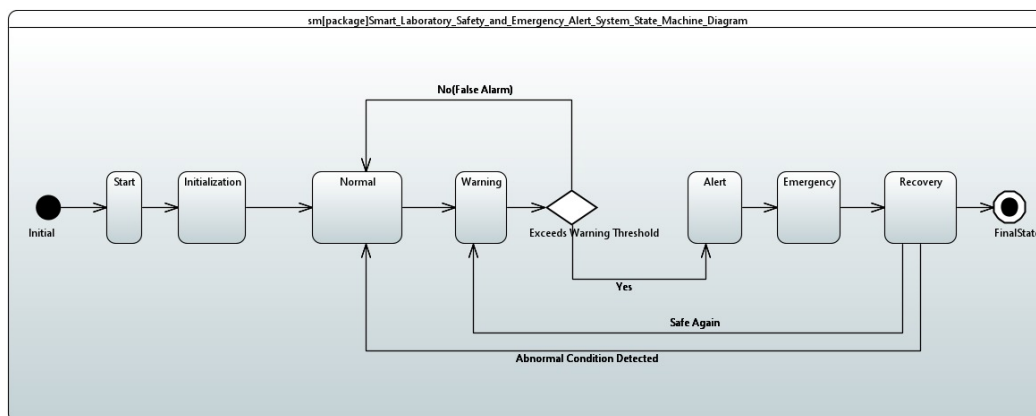


Figure 3: State Machine Diagram (system operational states and transitions)

Use Case Diagram

The Use Case Diagram captures all interactions between the User and the system. The primary use cases are: initiating the device, monitoring via active sensors and controllers

(which includes detecting temperature above threshold and detecting knock events), triggering alarms, disabling false alarms, and treating the cause of an alarm.

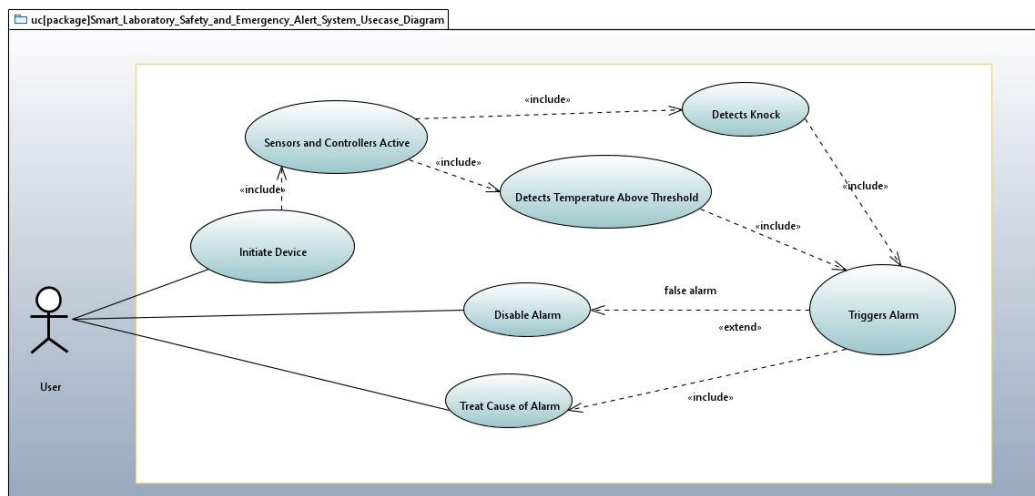


Figure 4: Use Case Diagram (user interactions with the system)

Sequence Diagram

The Sequence Diagram illustrates the end-to-end communication flow during operation. The ESP32 reads the sensors, determines the alert status, and simultaneously publishes the sensor packet to the Raspberry Pi MQTT broker and sends the status via BLE notification to the Arduino. Node-RED processes the incoming MQTT message, displays it on the dashboard, and can publish commands back to the ESP32 to update thresholds. The Arduino receives the BLE notification and controls the LED and buzzer.

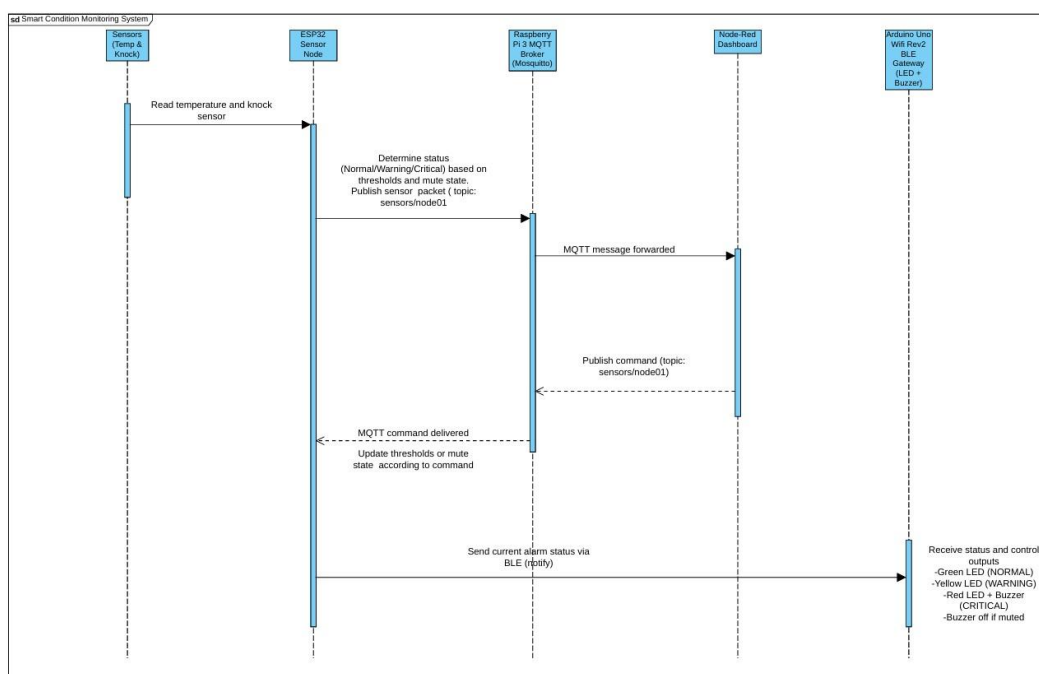


Figure 5: Sequence Diagram (BLE and MQTT communication flow)

6.1 ESP32 Sensor Node

The main loop runs every ~500 ms (limited by the knock sensor polling window). Each iteration: (1) reads temperature from GPIO34 via Steinhart-Hart; (2) polls GPIO32 for 500 ms counting knock events; (3) updates the rolling knock status with hold/decay logic; (4) evaluates the combined status as $\max(\text{tempStatus}, \text{knockStatus}, \text{forceAlert})$; (5) formats and publishes the MQTT packet; (6) updates and notifies the BLE characteristic.

Threshold validation on incoming CMD:CRIT commands: if the new critical value would be less than or equal to the current warning threshold, the warning threshold is automatically adjusted to critical – 1 to maintain a valid ordering.

6.2 Arduino Gateway

At startup the Arduino scans for the BLE service UUID. On finding ESP32_SensorNode it connects, discovers attributes, and subscribes to the characteristic notification. On each notification the first byte is checked: if 'M', the buzzer is muted and the second byte carries the status digit; otherwise the first byte is the status digit directly. LED colours: Green=NORMAL, Yellow=WARNING, Red=CRITICAL. Buzzer tones: 1 kHz / 300 ms for WARNING, 2 kHz / 800 ms for CRITICAL. On disconnect, the Arduino restarts scanning automatically (Blue LED).

6.3 Node-RED Dashboard

Five progressive flows were developed:

- Flow 01: MQTT in node subscribed to sensors/node01, output to debug sidebar. Verifies broker connectivity.
- Flow 02: Parse Packet function node splits semicolon-delimited fields. Feeds temperature gauge, status text, and knock count text widgets.
- Flow 03: fn_temp_chart and fn_knock_chart function nodes set msg.payload, msg.topic, and msg.x = Date.now() before feeding ui_chart nodes for history display.
- Flow 04: Remote control group — warning/critical threshold sliders (20–35°C), mute toggle switch, and test alert buttons. fn_build_cmd publishes CMD: strings to sensors/node01/cmd.
- Flow 05: fn_filter_alert passes only WARNING/CRITICAL status changes (30 s debounce). File node appends ISO timestamp + status + temperature to /home/pi/sensor_alerts.log.

7 Results

The system was demonstrated end-to-end with all three hardware nodes running simultaneously. Figure 6 shows the Node-RED dashboard during a live test session, capturing real sensor data and remote control in operation.

The following results were observed:

- Temperature accuracy: KY-013 read room temperature within $\pm 1^\circ\text{C}$ of a reference thermometer at approximately 23°C . Readings were stable with no filtering required at 12-bit resolution.
- Knock detection: KY-031 reliably detected single knocks with less than 50 ms latency. The WARNING threshold (≥ 2 knocks in 2 s) and CRITICAL threshold (≥ 5 knocks in 5 s) triggered correctly during testing.
- BLE link: Arduino connected to ESP32 within 5 seconds on first scan. After power-cycling either board, automatic reconnection completed within 8 seconds.
- MQTT throughput: Packets published every approximately 2 seconds with no observed packet loss on local WiFi. Node-RED dashboard updated within 1 second of each publish.
- Remote control: Warning and Critical threshold sliders updated ESP32 thresholds in real time, confirmed via Serial Monitor. Mute toggle silenced the buzzer while maintaining LED indication. Test alert buttons triggered the correct 5-second forced alert on the hardware.
- Alert logging: Flow 05 wrote timestamped entries to the log file with correct 30-second debounce. Log content was displayed in the dashboard Alert Log tab and verified via SSH on the Pi.

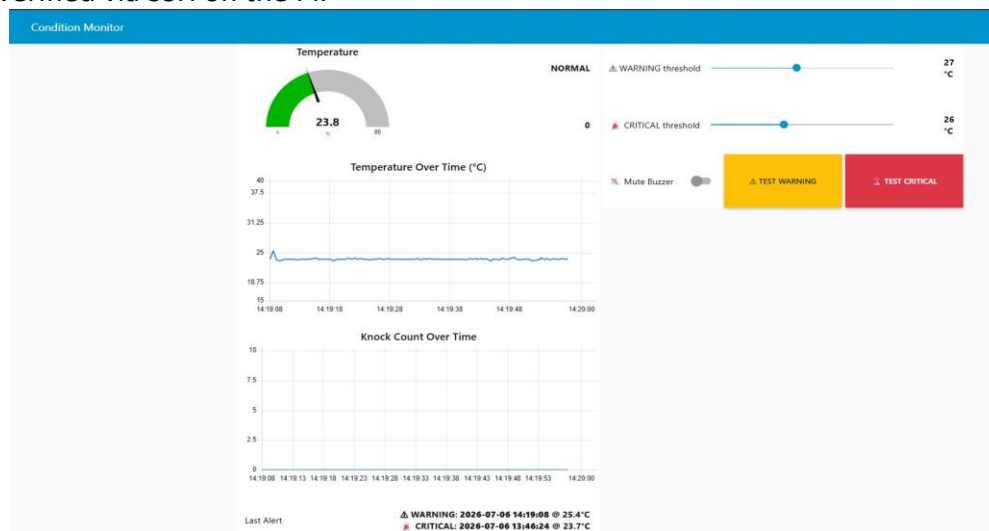


Figure 6: Node-RED dashboard - live temperature reading (23.8°C , NORMAL), temperature history chart, knock count chart, threshold sliders, mute toggle, test alert buttons, and last alert log entries

Limitations and Observations

- ESP32 flash: Enabling BLE + WiFi + MQTT simultaneously required the Huge APP (3 MB No OTA) partition scheme. The default partition scheme caused an out-of-memory linker error.
- Arduino flash: At 99% capacity with ArduinoBLE loaded. No additional libraries can be added without removing ArduinoBLE first.
- KY-013 accuracy: The NTC thermistor reading varies with supply voltage stability. A hardware low-pass filter or software averaging would improve accuracy in noisy power environments.

8 Sources / References

1. Espressif Systems. ESP32 Technical Reference Manual. Available at: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
2. Arduino. ArduinoBLE Library Reference. Available at: <https://www.arduino.cc/reference/en/libraries/arduinoBLE/>
3. Eclipse Foundation. Mosquitto MQTT Broker Documentation. Available at: <https://mosquitto.org/documentation/>
4. Node-RED Project. Node-RED Documentation. Available at: <https://nodered.org/docs/>
5. node-red-dashboard contributors. Dashboard nodes for Node-RED. Available at: <https://github.com/node-red/node-red-dashboard>
6. Knolleary. PubSubClient — MQTT library for Arduino/ESP. Available at: <https://github.com/knolleary/pubsubclient>
7. Steinhart, J.S. & Hart, S.R. (1968). Calibration curves for thermistors. Deep-Sea Research, 15(4), 497-503.
8. HSHL Hamm-Lippstadt University of Applied Sciences. Advanced Embedded Systems Lab Guide, 2026.